

Agentic DevOps

Building Agentic Skills for Infrastructure Automation

Version: 2026 Edition

Trainer : Gourav Shah

Duration: 3 Full Days

Level: Advanced

Delivery: Concepts + Guided Hands-On Projects

Objective

This workshop enables DevOps and SRE teams to adopt AI across three levels — leveraging platform AI already in their stack, using AI coding agents for infrastructure automation, and building custom agentic systems for operational workflows.

Participants will learn how to:

- Understand how LLMs work and where agents fit in the DevOps lifecycle
- Leverage platform AI features (AWS, Datadog, Grafana) and identify gaps they leave behind
- Use AI coding agents to generate production-grade Infrastructure as Code
- Write Agentic Skills — machine-readable runbooks for humans and agents
- Build working agents with YAML-first definitions
- Wire agents to real infrastructure using CLI tools and MCP
- Orchestrate multi-agent squads for cross-domain operations
- Connect agents to Slack, cron schedules, and webhooks for production use
- Implement governance, approval workflows, and audit trails

The emphasis is on building practical, enterprise-safe skills — not learning a specific tool or framework.

Three Pillars

1. **Platform AI** — Activate and evaluate AI features built into AWS, Datadog, Grafana, and other tools you already use
2. **AI Coding Agents** — Use structured workflows with OpenCode (or equivalent coding agents) to generate and validate Infrastructure as Code
3. **Custom Agentic Automation** — Design, build, and govern your own AI agents for operational workflows

Workshop Philosophy

Agents do not replace DevOps engineers.

They augment:

- Detection and monitoring
- Analysis and correlation
- Recommendation generation
- Proposal and remediation drafting

Humans retain:

- Approval authority
- Risk ownership
- Execution control
- Governance

Mandatory Prerequisites

Technical:

- Strong foundations in DevOps practices
- Working knowledge of at least one IaC tool (Terraform, Ansible, or Kubernetes manifests)
- Familiarity with CLI tools (kubectl, aws cli, psql)
- Git and pull request workflows

AI Coding Agent (Mandatory):

Participants must have access to at least one AI coding agent.

Preferred (Pick any one):

- Claude Code (with Claude Pro or Team subscription)
- OpenAI Codex CLI (with ChatGPT Plus or Team subscription)
- OpenCode (provider-agnostic, works with Google AI Studio free tier, Claude, OpenAI, or any hosted LLM)

AI Provider Access (Mandatory):

Participants must have access to a hosted AI provider for LLM-powered labs.

Preferred (Pick any one):

- Claude subscription
- OpenAI
- Gemini / Google AI Studio (free tier available)

- Equivalent hosted provider

Local-only models are not sufficient for this workshop.

System Requirements

- AWS account (free tier sufficient)
- EC2 instance (t3.micro) or equivalent Linux environment
- RDS PostgreSQL instance (db.t3.micro) for database labs
- Kubernetes cluster (KIND or managed cluster such as EKS)
- GitHub repository
- kubectl, Terraform, and Ansible
- Docker
- AI coding agent (Claude Code, OpenAI Codex, or OpenCode)
- Observability stack (CloudWatch, Datadog, or Grafana)
- Optional: OpenAgentix or equivalent agentic automation framework (for Day 3 labs)

Workshop Structure

The workshop follows a progressive build model across three days:

Day 1: Understand — AI foundations, platform AI capabilities, agent architecture, impact assessment

Day 2: Build with AI — Structured AI coding, Infrastructure as Code, agentic skills, tool integration

Day 3: Build Agents — Agent design patterns, domain-specific agent projects, fleet orchestration, triggers, governance

Participants progressively build across the three pillars:

Platform AI Discovery → AI-Assisted Coding → Agent Skills → Tool Integration → Agent Build → Fleet Orchestration → Triggers → Governance → Capstone

Day 1 – AI Foundations and Agent Architecture

Module 1 – AI Foundations for Operations Teams

Topics

- How LLMs work — tokens, context windows, temperature — through operational analogies
- The Spectrum: Chat → Copilot → Agent → Squad
- Agent anatomy: Brain (LLM) + Skills (runbooks) + Tools (CLI/MCP) + Guardrails (approvals)
- Why agents need guardrails and structured workflows

Hands-On Lab

Participants use OpenCode to interact with an LLM using real operational data (CloudWatch alarm JSON). Progressive prompt engineering exercise comparing output quality across different prompt structures.

Deliverable: Optimized prompt template for operational diagnosis.

Module 2 – Platform AI — Features Already in Your Stack

Topics

- AWS AI services: Q Developer, DevOps Guru, Predictive Scaling, Cost Anomaly Detection, RDS Performance Insights
- Observability AI: Datadog Watchdog, Grafana Sift
- What platform AI does well and where it falls short
- The gap between platform AI and custom agents

Hands-On Lab

Participants enable and explore platform AI features on their own AWS infrastructure:

- RDS Performance Insights with load generation
- Cost Explorer analysis
- CloudWatch anomaly detection
- Q Developer for query explanation

Deliverable: Written assessment of platform AI capabilities and gaps for their environment.

Module 3 – From Platform AI to Custom Agents

Concept

Why platform AI alone is insufficient — it is vendor-locked, cannot chain tools, and lacks organizational context.

Topics

- What custom agents add: skills, multi-tool workflows, memory, human-in-the-loop
- Agent definition approaches: YAML-first, code-first, and hybrid
- Core resource types: Agent (single) → Fleet (team) → Flow (workflow)
- Live walkthrough of a minimal agent definition
- Landscape of agentic automation frameworks

Hands-On Demo

Participants run their first agent — a minimal YAML definition — and observe how it processes an operational query.

Deliverable: First working agent running on participant's environment.

Module 4 – Impact Assessment — What Should We Automate?

Topics

- Evaluating automation candidates — not everything should be an agent
- The Automation Quadrant: frequency × complexity
- How to scope an agent project: start with the smallest useful thing

Hands-On Lab

Team exercise where participants:

- List their top 10 repetitive operational tasks
- Score each on frequency, time, error risk, and tool count
- Plot on the Automation Quadrant
- Select top automation candidate for Day 3 build project

Deliverable: Prioritized automation candidate list. Top pick becomes their Day 3 capstone project.

Day 2 – AI-Powered Automation and Agent Building

Module 5 – Structured AI Coding with OpenCode

Topics

- Why unstructured prompting produces inconsistent infrastructure
- The Superpowers workflow: Brainstorm → Design → Blueprint → Implement → Validate
- GSD (Get Shit Done) spec-driven workflow for larger projects
- Provider-agnostic usage: works with any LLM

Hands-On Project

Participants use the Superpowers workflow to build a complete Ansible playbook for EC2 hardening. Each phase is explicit — brainstorm approaches, design structure, blueprint specs, implement code, validate output.

Deliverable: A validated Ansible playbook created through the structured 5-step workflow.

Module 6 – AI-Assisted Infrastructure as Code

Topics

- Using AI coding agents for production-quality Terraform, Ansible, and Kubernetes manifests
- The GSD workflow for multi-file IaC projects
- Reviewing and validating AI-generated infrastructure

- Common AI failure modes in infrastructure generation

Hands-On Project

Participants pick one track and build real IaC for their lab environment:

- Track A (Terraform): RDS PostgreSQL module with CloudWatch alarms and SNS notifications
- Track B (Ansible): PostgreSQL client setup with monitoring agents and backup scripts
- Track C (Kubernetes): Application deployment with HPA, resource limits, and PodDisruptionBudget

Deliverable: Production-quality IaC artifact, validated and applicable to participant's environment.

Module 7 – Agent Skills — Teaching Agents Your Runbooks

Core Topic

Agentic Skills are structured, machine-readable runbooks that both humans and AI agents can follow.

Topics

- What is a Skill: structured workflow with defined inputs, tool sequences, decision trees, and output schemas
- The SKILL.md format: metadata, steps, commands, escalation rules
- Skill lifecycle: Design → Validate → Version → Deploy → Improve
- How skills connect to agents and automation frameworks

Hands-On Project

Using OpenCode, participants write a domain-specific SKILL.md:

- SRE: EC2 health check skill
- DevOps: Deployment safety check skill
- DBA: RDS slow query investigation skill
- Observability: Alert noise analyzer skill

Each skill must define required inputs, at least 5 CLI commands, a decision table, and escalation criteria.

Deliverable: Complete SKILL.md file with peer review.

Module 8 – Wiring Tools to Agents

Concept

Tools give agents the ability to act. Safety boundaries determine what they are allowed to do.

Topics

- Three tool integration patterns: Direct CLI, CLI wrappers with safety checks, MCP servers
- Tool configuration: shell, kubectl, aws, file, http, and MCP tools

- Safety configuration: allowed commands, blocked commands, credential protection
- When to use CLI vs MCP
- Writing custom tool wrappers

Hands-On Project

Participants build their first real agent with tools connected to their infrastructure:

- Define agent YAML with model, instructions, and tool access
- Configure allowed and blocked command lists
- Attach their Module 7 skill
- Test agent against live lab environment

Deliverable: Working agent YAML with tools, safety rules, and custom skill. Agent runs successfully against participant's infrastructure.

Day 3 – Agent Systems, Deployment, and Governance

Module 9 – Agent Design Patterns

Topics

- Single-agent patterns: Advisor → Investigator → Proposal → Guardian
- Multi-agent patterns: Sequential pipeline, Parallel investigation, Hierarchical (commander + specialists)
- The Autonomy Spectrum: L1 Assistive → L2 Advisory → L3 Proposal → L4 Semi-autonomous
- Mapping patterns to resource types: Agent, Fleet, Flow
- Reference architecture walkthrough of a production multi-agent system

Module 10 – Build Project — Your Domain Agent

Core Topic

Participants build a complete, production-ready agent for a real operational problem identified on Day 1.

Three guided tracks:

Track A — Database Health and Tuning Agent Connects to participant's RDS PostgreSQL, analyzes performance, identifies tuning opportunities, proposes parameter changes with approval gates.

Track B — Cost Anomaly and FinOps Agent Queries AWS Cost Explorer, detects unusual spending, identifies optimization opportunities, generates daily reports with Slack notifications.

Track C — Kubernetes Health and Self-Healing Agent Monitors Kind cluster for pod issues, diagnoses root cause, proposes fixes with approval before any mutations.

Build steps for all tracks:

1. Design agent spec using Superpowers workflow
2. Write the Agent YAML with tools and skills
3. Write domain-specific skills
4. Test against live infrastructure
5. Wrap in a Flow with triggers and conditions
6. Add governance layer with approval gates and audit logging

Deliverable: Complete agent package — agent.yaml, skills, flow.yaml, and governance.yaml.

Module 11 – Fleet Orchestration — Agents Working Together

Concept

Coordinated reasoning among specialized agents.

Topics

- When single agents are insufficient
- Fleet coordination modes: round-robin, skill-based, hierarchical
- The manager pattern: coordinator delegates to specialists
- Shared memory and tool access across a fleet

Hands-On Project

Teams of 3 combine their Module 10 agents into a coordinated fleet. Test with a cross-domain incident that requires database, infrastructure, and cost analysis.

Demonstration: How fleet orchestration works in practice using a reference agentic framework.

Deliverable: Fleet YAML coordinating 3 agents from different tracks.

Module 12 – Triggers, Scheduling, and Interface

Concept

Agents need production interfaces — CLI alone is not sufficient for enterprise use.

Topics

- Four interface patterns: CLI (development), Slack/Teams (team), Webhooks (automation), Cron (scheduled)
- Trigger systems: Slack, PagerDuty, GitHub, Jira, Cron, Webhook
- Mission Control: dashboard concept for managing deployed agents

Hands-On Project

Participants write trigger specifications:

- Cron schedule for daily health checks
- Slack commands for on-demand operations
- Webhook endpoint for alert-triggered workflows

Deliverable: Trigger YAML with at least one cron schedule and one Slack command.

Module 13 – Governance — Making Agents Enterprise-Safe

Topics

- The governance triad: what agents can DO × what they need APPROVAL for × what gets LOGGED
- Agent maturity levels:
 - L1 Assistive
 - L2 Advisory
 - L3 Proposal-driven
 - L4 Semi-autonomous
- Promotion criteria: how agents earn more autonomy over time
- Enterprise requirements: audit trails, RBAC, credential protection, rollback plans

Hands-On Project

Participants add a complete governance layer:

- Maturity level assignment with promotion criteria
- Approval workflows with channel routing and timeouts
- Credential scanning and redaction patterns
- Audit logging configuration
- Rollback policies

Deliverable: Governance YAML that passes enterprise security review standards.

Module 14 – Capstone — Demo and 30-Day Plan

Teams present their complete stack:

- The operational problem being solved
- Agent YAML + skills + tools
- Live demo against lab environment
- Flow with triggers and conditions
- Governance spec with approval mode and maturity level

Each team drafts a 30-day deployment roadmap:

- Week 1: Deploy to staging in observe-only mode

- Week 2: Review logs, tune skills, fix false positives
- Week 3: Promote to advisory mode, enable Slack notifications
- Week 4: Review metrics, prepare promotion proposal

Deliverable: Written 30-day deployment plan.

Final Capstone

Teams present a complete Agentic DevOps system including:

- Defined agent roles and profiles
- Domain-specific agentic skills
- Tool integration with safety boundaries
- Multi-agent fleet orchestration
- Trigger and scheduling configuration
- Approval and governance framework
- 30-day production deployment roadmap

Key Differentiators

This workshop does not teach ad-hoc prompting or theoretical agent design.

It teaches:

- Three-pillar approach: platform AI, AI coding agents, and custom agentic automation
- Structured AI coding workflows (Superpowers and GSD)
- Agentic Skills as machine-readable runbooks
- Tool-agnostic agent definitions that work across frameworks
- Progressive build model — understand, then build, then deploy
- Enterprise governance from day one
- Real infrastructure, real tools, real agents
- Every participant takes home a working agent system

#corp/outlines/2026